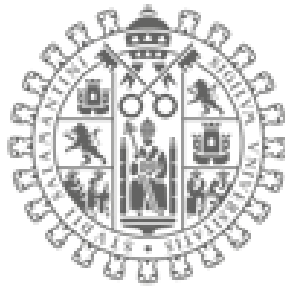


Calidad y Auditoría

Detección de errores, defectos y fallos de software



**VNiVERSiDAD
D SALAMANCA**

CAMPUS OF INTERNATIONAL EXCELLENCE

Miguel Barranquero Diez

Índice

1. Introducción	3
2. Programa 1	4
3. Programa 2	8
4. Programa 3	13
5. Programa 4	14
6. Programa 5	15
Bibliografía	16

1. Introducción

El presente trabajo se centra en el análisis y la corrección de defectos en cinco programas software proporcionados. La estructura de este ejercicio se basa en la aplicación de una metodología de prueba y depuración específica para cada código fuente. Esta metodología implica:

- Identificar defectos.
- Identificar un caso de prueba que dé lugar a un fallo.
- Si es posible, identificar el caso de prueba que no dé lugar a un fallo.
- Si es posible, identificar el caso de prueba no produzca un estado de error.
- Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo.
- Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

2. Programa 1

La Figure 1 corresponde con el primer programa a analizar:

```
public class Array1 {

    public static void main(String[] args) {

        //Devuelve el número de ocurrencias de 0 en x

        int x[]={1,0,0};
        int count=0;
        for (int i=1 ; i<x.length; i++)
        {
            if (x[i] == 0)
            {
                count ++;
            }
        }
        System.out.print ("Número de ceros: ");
        System.out.println(count);
    }

}
```

Figure 1: Programa 1

Identificar defectos (sin ejecutar el código)

El defecto del código es que el bucle for comienza desde el índice 1 ($i=1$), es decir, en el segundo elemento del array. Este debería de empezar en el índice 0 ($i=0$) para poder considerar y analizar todos los elementos del array.

```
for (int i=1 ; i<x.length; i++)
```

Figure 2: Programa 1 - Defecto

Identificar un caso de prueba que dé lugar a un fallo

Para realizar el caso de prueba que de lugar a fallo introduciré en la primera posición, ($i=0$ del array) un 0, este cero no se tendrá en cuenta por lo que el número de 0 esperado será incorrecto.

Para este ejemplo se ha declarado el array $x[] = \{0,0,0\}$ (Figure 3):

-Número de 0 esperado: 3.

-Número de 0 obtenido: 2.

```

1 public class Array1 {
2     public static void main(String[] args) {
3         //Devuelve el número de ocurrencias de 0 en x
4         int x[]={0,0,0};
5         int count=0;
6         for (int i=1 ; i<x.length; i++){
7             if (x[i] == 0){
8                 count ++;
9             }
10        }
11        System.out.print ("Número de ceros: ");
12        System.out.println(count);
13    }
14 }
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array1
Número de ceros: 2
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 3: Programa 2 - Fallo Ejemplo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Para conseguir este caso de prueba bastará con que en la primera posición ($i=0$) del array no exista un 0, el defecto sigue estando pero con esta disposición (evitar 0 en la primera posición) la ejecución no dará fallo.

Para este ejemplo se ha declarado el array $x[] = \{1,0,0\}$ (Figure 4):

-Número de 0 esperado: 2.

-Número de 0 obtenido: 2.

```

1 public class Array1 {
2     public static void main(String[] args) {
3         //Devuelve el número de ocurrencias de 0 en x
4         int x[]={1,0,0};
5         int count=0;
6         for (int i=1 ; i<x.length; i++){
7             if (x[i] == 0){
8                 count ++;
9             }
10        }
11        System.out.print ("Número de ceros: ");
12        System.out.println(count);
13    }
14 }
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array1
Número de ceros: 2
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 4: Programa 1 - No Fallo Ejemplo

Si es posible, identificar el caso de prueba no produzca un estado de error

Para este caso de prueba será necesario que el array no tenga ningún 0, de esta manera aunque no se analice el primer elemento, no será relativo para determinar cuantos 0 hay en el array puesto que no existirá ninguno haciendo que no se produzca el estado de error.

Para este ejemplo se ha declarado el array $x[] = \{1,2,3\}$ (Figure 5):

-Número de 0 esperado: 0.

-Número de 0 obtenido: 0.

```

1
2 public class Array1 {
3     public static void main(String[] args) {
4         //Devuelve el número de ocurrencias de 0 en x
5         int x[]={1,2,3};
6         int count=0;
7         for (int i=1 ; i<x.length; i++){
8             if (x[i] == 0){
9                 count ++;
10            }
11        }
12        System.out.print ("Número de ceros: ");
13        System.out.println(count);
14    }
15 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array1
Número de ceros: 0
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 5: Programa 1 - No Fallo Ejemplo

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Para este caso sera necesario que en el primer elemento nunca sea 0 para evitar el fallo, pero el estado de error seguirá ahí ya que no se evalúa la condición (cero/noCero) en el primer elemento (i=0).

Para este ejemplo se ha declarado el array $x[] = \{99,0,0\}$ (Figure 6):

-Número de 0 esperado: 2.

-Número de 0 obtenido: 2.

```

1
2 public class Array1 {
3     public static void main(String[] args) {
4         //Devuelve el número de ocurrencias de 0 en x
5         int x[]={99,0,0};
6         int count=0;
7         for (int i=1 ; i<x.length; i++){
8             if (x[i] == 0){
9                 count ++;
10            }
11        }
12        System.out.print ("Número de ceros: ");
13        System.out.println(count);
14    }
15 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array1
Número de ceros: 2
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 6: Programa 1 - No Fallo Ejemplo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

Para corregir el defecto de cambia la línea 7:

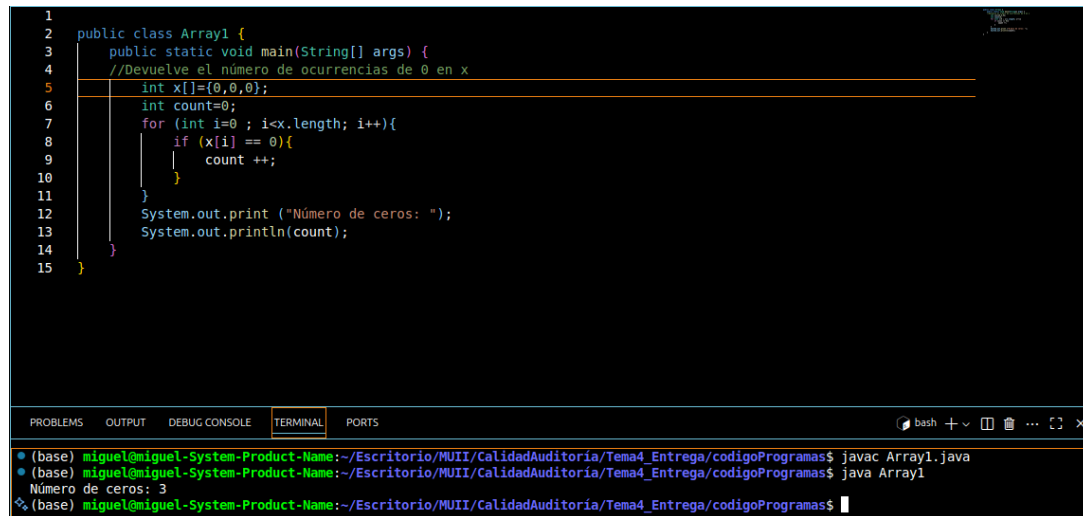
-Defecto: `for (int i=1 ; i < x.length; i++)`

-Corregido: `for (int i=0 ; i < x.length; i++)`

El caso de prueba del punto 2 correspondía con el siguiente array $x[] = \{0,0,0\}$ (Figure 7), la ejecución con el defecto corregido:

-Número de 0 esperado: 3.

-Número de 0 obtenido: 3.



```

1
2 public class Array1 {
3     public static void main(String[] args) {
4         //Devuelve el número de ocurrencias de 0 en x
5         int x[]={0,0,0};
6         int count=0;
7         for (int i=0 ; i<x.length; i++){
8             if (x[i] == 0){
9                 count ++;
10            }
11        }
12        System.out.print ("Número de ceros: ");
13        System.out.println(count);
14    }
15 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array1
Número de ceros: 3
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 7: Programa 1 - No Fallo No Defecto

3. Programa 2

La Figure 8 corresponde con el segundo programa a analizar:

```
public class Array2 {
    public int findLast (int[] x, int y) {
        //Devuelve el índice del último elemento en x que sea igual a
        //Si no existe dicho elemento devuelve -1

        for (int i=x.length-1;i>0; i--)
        {
            if (x[i] == y)
            {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        int x[]={0,1,1};
        int y=1;

        Array2 a=new Array2();
        int resultado = a.findLast (x,y);
        System.out.print ("El índice del elemento buscado es: ");
        System.out.print (resultado);
    }
}
```

Figure 8: Programa 2

Identificar defectos (sin ejecutar el código)

El defecto del código está en la condición del bucle for en la línea 5, El bucle se detiene cuando $i > 0$, lo que significa que nunca evalúa el elemento en la posición 0 (el primer elemento del array).

```
for (int i=x.length-1;i>0; i--)
```

Figure 9: Programa 2 - Defecto

Identificar un caso de prueba que dé lugar a un fallo

Para producir un fallo, el elemento buscado (y) debe estar únicamente en la posición 0 del array.

Para este ejemplo se ha declarado el array $x[] = \{1,6,7\}$ siendo $y = 1$ (Figure 10):

-Número de índice esperado: 0.

-Número de índice obtenido: -1.

```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el índice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i > 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10        return -1;
11    }
12
13
14
15
16    public static void main(String[] args) {
17        int x[]={1,6,7};
18        int y=1;
19        Array2 a=new Array2();
20        int resultado = a.findLast (x,y);
21        System.out.print("El índice del elemento buscado es: ");
22        System.out.println(resultado);
23    }
24 }
25

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El índice del elemento buscado es: -1
❖ (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 10: Programa 2 - Fallo

El programa no encuentra el elemento que sí existe en la posición 0

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Para evitar el fallo, el elemento buscado no debe estar en la posición 0, o debe estar también en otras posiciones.

Para este ejemplo se ha declarado el array $x[] = \{4,6,1\}$ siendo $y = 1$ (Figure 11):

-Número de índice esperado: 2.

-Número de índice obtenido: 2.

```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el índice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i > 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10        return -1;
11    }
12
13
14
15
16    public static void main(String[] args) {
17        int x[]={4,6,1};
18        int y=1;
19        Array2 a=new Array2();
20        int resultado = a.findLast (x,y);
21        System.out.print("El índice del elemento buscado es: ");
22        System.out.println(resultado);
23    }
24 }
25

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El índice del elemento buscado es: 2
❖ (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 11: Programa 2 - No Fallo

No hay fallo pero el defecto sigue presente.

Si es posible, identificar el caso de prueba no produzca un estado de error

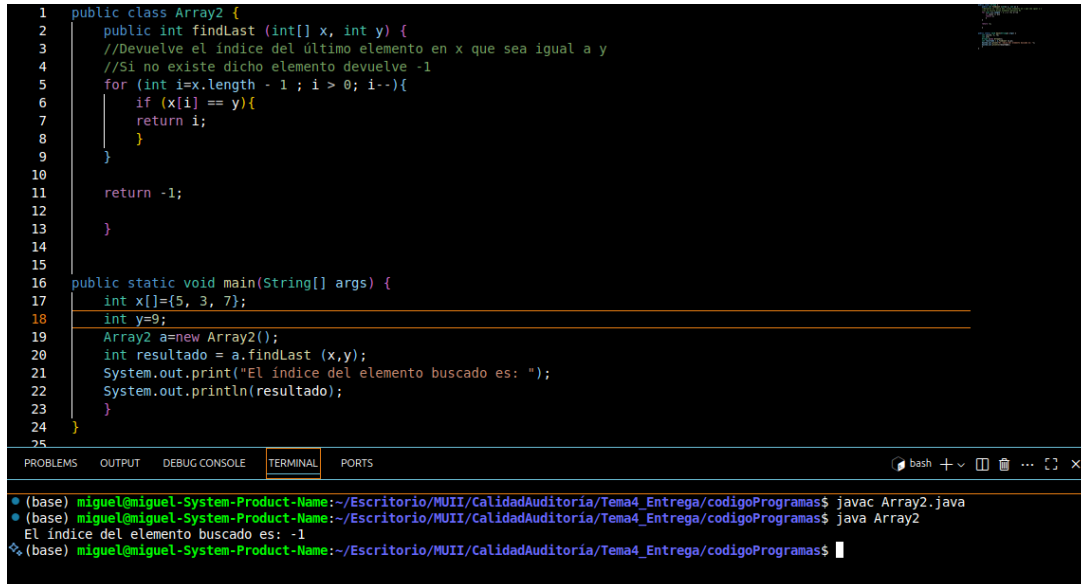
Para que no haya estado de error, el bucle debe funcionar correctamente sin necesidad de evaluar la posición 0. Esto ocurre cuando:

-El elemento buscado está en posiciones > 0 , O -El elemento buscado no existe en el array

Para este ejemplo se ha declarado el array `int x[] = {5, 3, 7}` siendo `y = 9` (Figure 12):

-Número de índice esperado: -1.

-Número de índice obtenido: -1.



```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el índice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i > 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10
11         return -1;
12     }
13 }
14
15
16 public static void main(String[] args) {
17     int x[]={5, 3, 7};
18     int y=9;
19     Array2 a=new Array2();
20     int resultado = a.findLast (x,y);
21     System.out.print("El índice del elemento buscado es: ");
22     System.out.println(resultado);
23 }
24
25

```

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El índice del elemento buscado es: -1
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 12: Programa 2 - No Error

NO hay estado de error porque el elemento no existe y no importa que no se revise la posición 0

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Para este caso, el elemento buscado debe estar en la posición 0, pero también en otras posiciones posteriores. Así, aunque no se evalúe la posición 0 (estado de error), el método encuentra el elemento en otra posición y devuelve un resultado “aceptable” (aunque no sea técnicamente el último).

Para este ejemplo se ha declarado el array `int int x[] = {1, 0, 1}` siendo `y = 1` (Figure 13):

-Número de índice esperado: 2.

-Número de índice obtenido: 2.

```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el indice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i > 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10
11         return -1;
12     }
13 }
14
15
16 public static void main(String[] args) {
17     int x[]={1, 0, 1};
18     int y=1;
19     Array2 a=new Array2();
20     int resultado = a.findLast (x,y);
21     System.out.print("El indice del elemento buscado es: ");
22     System.out.println(resultado);
23 }
24 }
25

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El indice del elemento buscado es: 2
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 13: Programa 2 - Error No Fallo

Si hay un estado de error puesto que no revisa la posición 0 del array pero no se produce un fallo, pero no hay fallo porque encuentra el elemento en posición 2, que efectivamente es la última ocurrencia válida.

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

Para corregir el defecto se cambia la línea 5:

-Defecto: for (int i = x.length - 1 ; i > 0; i--)

-Corregido: for (int i = x.length - 1 ; i >= 0; i--)

El caso de prueba del punto 2 correspondía con el siguiente array $x[] = \{1, 6, 7\}$ y $y = 1$ (Figure 14), la ejecución con el defecto corregido:

-Número de índice esperado: 0 .

-Número de índice obtenido: 0.

```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el índice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i >= 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10
11         return -1;
12     }
13 }
14
15
16 public static void main(String[] args) {
17     int x[]={1, 6, 7};
18     int y=1;
19     Array2 a=new Array2();
20     int resultado = a.findLast (x,y);
21     System.out.print("El índice del elemento buscado es: ");
22     System.out.println(resultado);
23 }
24 }
25

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El índice del elemento buscado es: 0
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 14: Programa 2 - No Fallo No Defecto

4. Programa 3

La Figure 15 corresponde con el tercer programa a analizar:

```
public class Array3 {
    public int lastZero (int[] x) {
        //Devuelve el índice del último 0 en x
        //Si no existe 0 devuelve -1

        for (int i=0; i<x.length-1; i++)
        {
            if (x[i] == 0)
            {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        int x[]={0,1,0};

        Array3 a=new Array3();
        int resultado = a.lastZero (x);
        System.out.print ("El índice del último 0 es: ");
        System.out.print (resultado);
    }
}
```

Figure 15: Programa 3

Identificar defectos (sin ejecutar el código)

Identificar un caso de prueba que dé lugar a un fallo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Si es posible, identificar el caso de prueba no produzca un estado de error

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

5. Programa 4

La Figure 16 corresponde con el cuarto programa a analizar:

```
public class Array4 {  
    public int countPositive (int[] x) {  
        //Devuelve el número de elementos positivos en x  
        int count=0;  
        for (int i=0; i<x.length; i++)  
        {  
            if (x[i] >= 0)  
            {  
                count++;  
            }  
        }  
        return count;  
    }  
  
    public static void main(String[] args) {  
  
        int x[]={-4,2,0,2};  
  
        Array4 a=new Array4();  
        int resultado = a.countPositive (x);  
        System.out.print ("El número de elementos positivos es: ");  
        System.out.print (resultado);  
    }  
}
```

Figure 16: Programa 4

Identificar defectos (sin ejecutar el código)

Identificar un caso de prueba que dé lugar a un fallo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Si es posible, identificar el caso de prueba no produzca un estado de error

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

6. Programa 5

La Figure 17 corresponde con el quinto programa a analizar:

```
public class Array5 {
    public int oddOrPositive (int[] x) {
        //Devuelve el número de elementos en x que son impares o positivos
        int count=0;
        for (int i=0; i<x.length; i++)
        {
            if (x[i]%2 == 1 || x[i]>0)
            {
                count++;
            }
        }
        return count;
    }

    public static void main(String[] args) {

        int x[]={-4,2,1,2};

        Array5 a=new Array5();
        int resultado = a.oddOrPositive (x);
        System.out.print ("El número de elementos impares o positivos es: ");
        System.out.print (resultado);
    }
}
```

Figure 17: Programa 5

Identificar defectos (sin ejecutar el código)

Identificar un caso de prueba que dé lugar a un fallo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Si es posible, identificar el caso de prueba no produzca un estado de error

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

Bibliografía